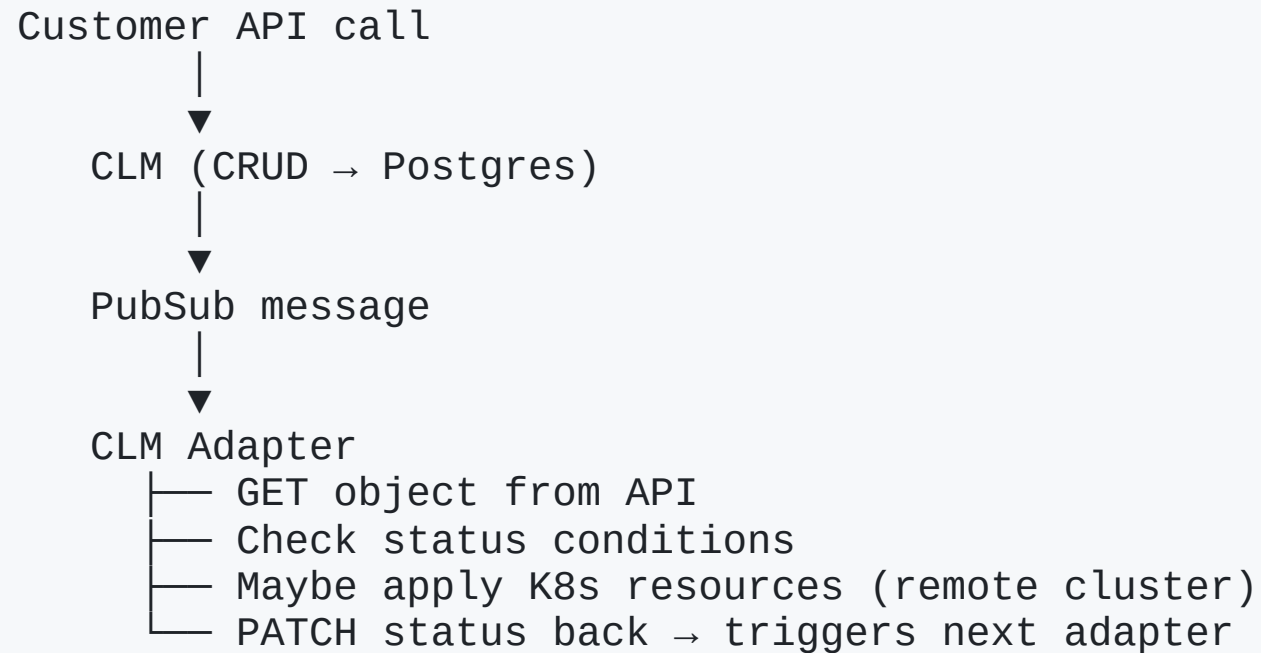


storectrl

Kube-native controllers, any datastore

Where we are today — CLM



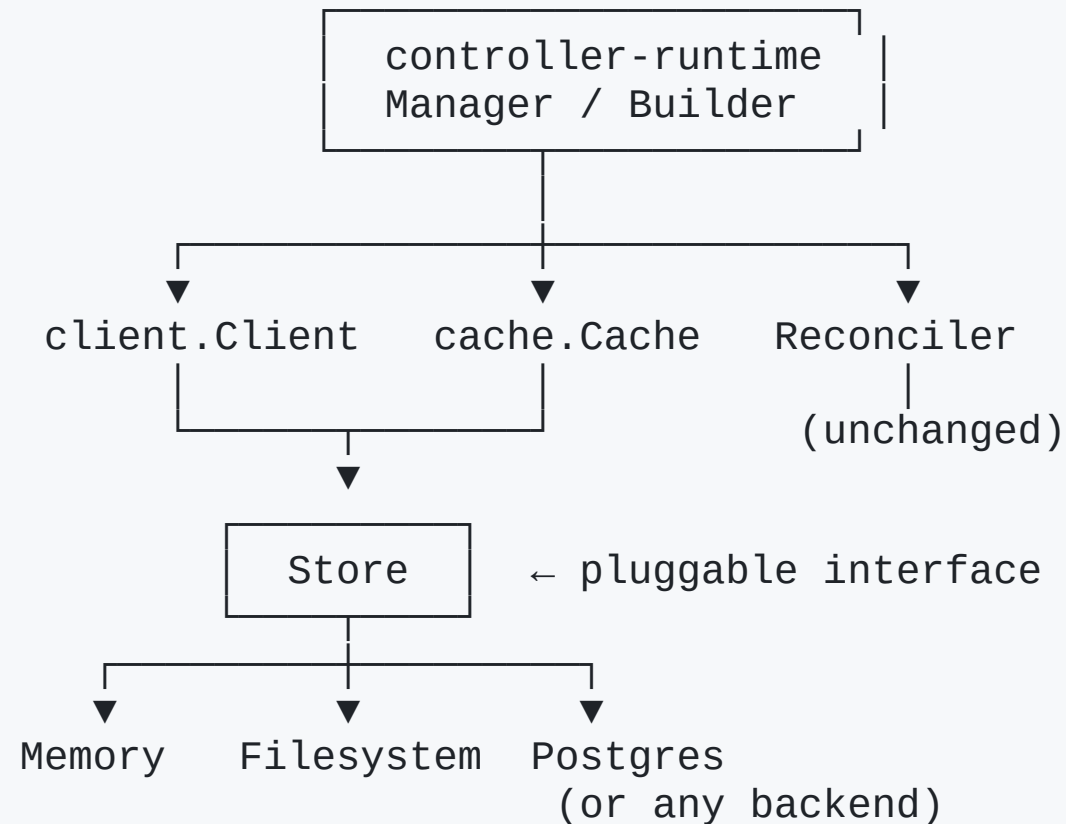
Each adapter is a step in a chain. State flows through status fields. Ordering, retries, error handling — all hand-rolled per adapter.

What we want

- **Controller pattern** — watch + reconcile, not event chains
- **Optimistic concurrency** — built-in conflict detection
- **Familiar tooling** — controller-runtime, same patterns every team knows
- **Not etcd** — business data belongs in a real database (backups, PITR, SQL queries, audit)
- **Pluggable backends** — postgres today, maybe a GCP API tomorrow

The idea

What if we could write standard controller-runtime reconcilers that read and write from a pluggable datastore instead of etcd?



Store interface — intentionally simple

```
type Store interface {  
    Get(ctx, key, obj)                error  
    List(ctx, list, opts...)          error  
    Create(ctx, obj)                  error  
    Update(ctx, obj)                  error  
    Delete(ctx, obj)                  error  
    Watch(ctx, list, opts...)         (Watcher, error)  
    Apply(ctx, applyConfig, opts...)  error  
}
```

No Patch, no DeleteAllOf, no SubResources on Store.

Those are composed in the `client.Client` wrapper (Patch = Get + merge + Update, etc.).

Backend implementors only need these 7 methods.

What storectrl provides

Component	What it does
<code>Store</code> interface	7 methods a backend must implement
<code>NewClient(store)</code>	Returns <code>client.Client</code> — drop-in for controller-runtime
<code>NewCache(store)</code>	Returns <code>cache.Cache</code> — watch-backed, in-memory reads
<code>StoreListerWatcher</code>	Adapter to CR's full informer stack (Reflector, DeltaFIFO)
<code>BaseObject</code> / <code>BaseList</code>	Embed in your types for <code>client.Object</code> compatibility
Error types	<code>apierrors.NotFound()</code> etc. work — transparent swap
Conformance suite	<code>storetest.TestStore(t, store)</code> — validates any backend

Components are independent — use any combination, mix with CR defaults.

What a controller looks like

```
type Cluster struct {
    storectrl.BaseObject `json:",inline"`
    Spec                  ClusterSpec   `json:"spec"`
    Status                ClusterStatus `json:"status"`
}

// Standard controller-runtime reconciler. Zero storectrl-specific code.
func (r *ClusterReconciler) Reconcile(ctx context.Context,
    req ctrl.Request) (ctrl.Result, error) {

    var cluster Cluster
    if err := r.Client.Get(ctx, req.NamespacedName, &cluster); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    // ... reconcile logic, same as any kube controller
}
```

Wiring — swap two factories

```
store := postgres.NewShardedStore(db, scheme, shardID)

mgr, _ := ctrl.NewManager(ctrl.GetConfigOrDie(), ctrl.Options{
    // These two lines are the only difference from a standard setup
    NewClient: storectrl.NewClientFunc(store, scheme),
    NewCache:  storectrl.NewCacheFunc(store, scheme),
})

// Everything else is identical
ctrl.NewControllerManagedBy(mgr).
    For(&Cluster{}).
    Complete(&ClusterReconciler{})

mgr.Start(ctx)
```


What you keep from controller-runtime

Because storectrl plugs into the standard manager, you get all of this for free:

- **Leader election** via Kubernetes Leases
- **Metrics server** (Prometheus)
- **Health probes** (liveness / readiness)
- **Graceful shutdown**
- **Controller options** — MaxConcurrentReconciles, RateLimiter, etc.
- **Controller builder** — `For()`, `Owns()`, `Watches()`

You can also **mix and match**: override only `NewClient` or only `NewCache`, keep CR defaults for the other side (e.g., cache from Store, writes to API server).

Sharding is native: a Store can scope to a shard, so each controller instance reconciles only its own objects. The postgres backend does this with lease-fenced writes (rejects writes when the shard lease is lost).

Backends — anything that can store and notify

A backend can be a database, a filesystem, an API, a KV store... anything that fulfills the Store contract.

Backend	Example
Database	Postgres, MySQL, CockroachDB, Spanner
Document / NoSQL	Firestore, DynamoDB, MongoDB
Filesystem	JSON files on disk
API	GCP APIs, CLS API, any REST/gRPC service
KV store	Redis, Consul, etcd itself
In-memory	Tests, dev, single-process

Backend requirements

To implement `Store`, a backend must guarantee:

- **Thread safety** — concurrent reads and writes
- **Optimistic concurrency** — reject updates with stale `ResourceVersion`
- **Monotonic `ResourceVersion`** — numeric string, never decreases
- **Snapshot-consistent List** — all objects at a single point in time
- **Watch with resumption** — replay events from a given revision
- **Overflow → close** — close the watch on buffer full, never drop events silently
- **Deep copy** — callers may mutate after Create/Update
- **UID generation** — assign unique ID on Create

Optional: no-op suppression, server-side apply, revision persistence.

A conformance test suite (`storetest.TestStore`) validates all of this.

PoC backends today

Backend	Purpose	Watch support
Memory	Tests, dev, single-process	Event log with overflow → relist
Filesystem	Persistence without a DB, debugging	JSON files, same watch pattern
Postgres	Production path — sharded, lease-fenced	Via postgres-controller-backend

Adding a backend = implement 7 methods + run conformance suite.

What this is / what this is not

This is:

- An early PoC / spike
- Exploring feasibility of the approach
- A Go library, not a service

This is not:

- A replacement for CLM (yet)
- Production-ready
- A custom controller-runtime fork — it uses the real thing

Questions?